Alexandria University
Faculty of Engineering



Introduction to Machine Learning
Assigned: Saturday, Sep 26, 2026
Due: Saturday, Oct 3, 2026

Assignment 1

1. Introduction

In the lecture, K-Nearest Neighbors made predictions by finding the training points closest to a query point, usually using Euclidean distance, and letting those neighbors vote or average. This assignment applies the same core idea — find similar things, then let them inform a prediction — to a different and very widely used problem: recommending items to people.

The twist is in the title: closest is no longer measured with Euclidean distance. You will implement two other ways of measuring how similar two users (or two items) are — cosine similarity and Pearson correlation — and see for yourself why they tend to work better for this kind of data than plain distance does. The goal is not to memorize "use cosine for recommenders" as a rule; it's to understand, from the structure of the data, why Euclidean distance misleads you here and what each alternative actually fixes.

2. Learning Objectives

- Explain, in your own words, why Euclidean distance can be a poor similarity measure for rating data.
- Implement cosine similarity and Pearson correlation from first principles (no library shortcuts).
- Build both user-based and item-based collaborative filtering on top of the same similarity functions.
- Correctly handle missing data (most users haven't rated most items) when computing similarity.
- Evaluate a recommender quantitatively (RMSE / MAE) instead of by eyeballing results.

3. Background

3.1 Collaborative filtering, in one idea

Collaborative filtering recommends items to a user based purely on patterns in a ratings matrix — rows are users, columns are items, and each cell is a rating (or empty, if that user never rated that item). No information about the items themselves (genre, price, description) is used at all. The idea is: people who agreed in the past will probably agree again. If two users have rated many movies the same way, and one of them loved a movie the other hasn't seen, that movie is a good candidate to recommend to the second user.

There are two natural directions to apply this idea, and you will implement both:

- **User-based CF:** find users similar to the target user, then predict a rating as a weighted average of what those similar users rated the target item.
- **Item-based CF:** find items similar to the target item (based on how users rated them), then predict a rating as a weighted average of how the target user rated those similar items.

3.2 Why not just reuse Euclidean distance?

Consider two users who watch the exact same movies and always agree on which ones are better or worse than the others — but one of them is a generous rater who tends to give 4s and 5s, while the other is a harsh rater who gives 1s and 2s for the same relative opinions. Euclidean distance treats their

Dr. Ahmed El-Sayed

Eng. Youssef El-Ebiary Eng. Sohail Elganagy

Eng. Nour Mohamed Eng. Omar Abourida

rating vectors as far apart, because it only looks at the raw numeric gap between ratings. But their taste — the pattern of what they like relative to what they dislike — is identical. A good similarity measure should recognize that.

Worked example

User A (generous) rates four movies: `[5, 4, 5, 3]`

User B (harsh) rates the same four movies: `[3, 2, 3, 1]`

Notice B = A − 2 for every movie: identical relative opinions, different baseline.

Euclidean distance: $\sqrt{(2^2+2^2+2^2+2^2)} = 4$ → looks moderately far apart.

Cosine similarity: ≈ `0.987` → close to 1, but not exact — cosine corrects for scale, not for a constant offset.

Pearson correlation: = `1.0 exactly` → perfect similarity, because Pearson mean-centers each user's ratings before comparing, which removes the additive offset entirely.

Work through the arithmetic yourself before writing any code — it's four numbers per user. Understanding why Pearson gets exactly 1.0 here, and cosine doesn't, is the single most important thing this assignment is testing.

3.3 The three metrics, precisely

Let u and v be two rating vectors, restricted to only the items both have rated (more on this below).

```
Euclidean distance:   d(u,v) = √( Σᵢ (uᵢ − vᵢ)² )
                       similarity(u,v) = 1 / (1 + d(u,v))
Cosine similarity:    cos(u,v) = (u · v) / (‖u‖ · ‖v‖)
Pearson correlation:  pearson(u,v) = cos(u − ū, v − v̄)
                       i.e. mean-center each vector, then take cosine similarity
```

(Converting Euclidean distance into a similarity with 1/(1+d) is just so all three metrics point the same way: higher number = more similar. Without it you'd be mixing "lower is better" and "higher is better" when comparing methods later.)

3.4 The sparsity problem: co-rated items only

Almost every cell in a real ratings matrix is empty — a given user has rated a tiny fraction of all movies. When comparing user A and user B, you must restrict the comparison to items both of them actually rated. Treating a missing rating as 0 is a common and serious bug: it tells the similarity function that the user hated everything they haven't seen yet, which is not true and will silently wreck every similarity score. If two users (or two items) share fewer than 2 co-rated items, their similarity is undefined — skip them rather than guessing.

3.5 From similarity to a predicted rating

Once you have the k most similar users (or items) to your target, predict the missing rating as a similarity-weighted average — neighbors who are more similar should count for more:

```
predicted(u, i) = Σₙ sim(u,n) · rating(n,i)  /  Σₙ |sim(u,n)|
```

```
(sum over the k neighbors n that have rated item i)
```

4. Dataset: MovieLens (ml-latest-small)

You will use the MovieLens "latest-small" dataset, published by GroupLens Research at the University of Minnesota. It is free, open, and the standard teaching dataset for collaborative filtering — small enough to run comfortably on a laptop, real enough that the results mean something.

- **Size:** about 100,000 ratings from 600 users on 9,000 movies (ratings from 0.5 to 5.0 in 0.5 steps).
- **Download:** https://files.grouplens.org/datasets/movielens/ml-latest-small.zip (a ~1 MB zip file, no account or request needed).
- **Files you need:** ratings.csv (columns: userId, movieId, rating, timestamp) and movies.csv (columns: movieId, title, genres).

The dataset is provided for research and education under GroupLens's usage license — free to use for this assignment, but cite it in your write-up (F. Maxwell Harper and Joseph A. Konstan, "The MovieLens Datasets: History and Context," ACM TiiS, 2015) and don't redistribute the raw files.

If your machine can handle more and you want a meatier experiment for the bonus section, the larger ml-latest (full) dataset is on the same GroupLens page — but ml-latest-small is what the tasks below assume.

5. Setup

- Python 3.9+ with pandas and numpy. No scikit-learn, no Surprise, no other recommender library — see Section 8.
- Download and unzip ml-latest-small.zip into a data/ folder next to your notebook or script.
- A Jupyter notebook is the expected format, so you can show intermediate outputs and your written answers side by side with code.

6. Tasks

Part A — Load and explore (5 points)

1. Load ratings.csv and movies.csv into pandas DataFrames.
2. Report: number of unique users, number of unique movies, number of ratings, and the sparsity of the matrix (fraction of user–movie pairs that are NOT rated).
3. Build the user–item rating matrix: rows = userId, columns = movieId, values = rating, missing = NaN. (pandas' pivot_table is the natural tool here.)
4. Plot or print the distribution of ratings, and the distribution of how many ratings each user has given. What do you notice?

Part B — Similarity metrics from scratch (25 points)

Implement the following, working only over co-rated items/users (Section 3.4):

5. get_co_rated(matrix, a, b, axis) — returns the two aligned rating vectors for a and b (users if axis="index", items if axis="columns"), containing only entries where both have a rating.
6. euclidean_similarity(vec1, vec2) — Euclidean distance converted to a similarity via 1/(1+d).

7. cosine_similarity(vec1, vec2).
8. pearson_similarity(vec1, vec2).

Sanity-check every function against the worked example in Section 3.2 before moving on — you should get 1.0, 0.987, and 1.0 respectively.

Part C — User-based collaborative filtering (20 points)

9. find_k_nearest_users(matrix, target_user, k, similarity_fn) — the k most similar users to target_user, as a list of (user_id, score), sorted descending. Skip any user with fewer than 2 co-rated items with target_user.
10. predict_rating_user_based(matrix, target_user, target_item, k, similarity_fn) — a similarity-weighted average (Section 3.5) using only neighbors who actually rated target_item. Return None if none of the k neighbors rated it.

Part D — Item-based collaborative filtering (15 points)

11. find_k_nearest_items(matrix, target_item, k, similarity_fn) and predict_rating_item_based(matrix, target_user, target_item, k, similarity_fn) — the same logic as Part C, but neighbors are computed over items (columns) instead of users (rows).
12. In one or two sentences: which of your functions from Part C did you reuse here, and what did you have to change?

Part E — Evaluation (20 points)

13. train_test_split_ratings(ratings_df, test_fraction=0.2) — hold out 20% of EACH user's ratings for testing (not a random global split — a user with zero ratings left in training is useless as a test case).
14. evaluate(matrix, test_df, k, similarity_fn, method) — for every held-out (user, item, true_rating), predict a rating and compute RMSE and MAE over every prediction that was actually possible (some will return None — report what fraction that was, too).
15. Run the full grid: {user-based, item-based} × {euclidean, cosine, pearson} × k ∈ {5, 10, 20, 50}. Present the results as a table.

Part F — Analysis (15 points, written, no new code)

16. Based on your own results table, which similarity metric performed best, and does that match the intuition from Section 3.2? Where did it disagree, if anywhere?
17. Construct your own small numeric example (2 users, 3-4 items) — different from the one in Section 3.2 — where Euclidean-based similarity and Pearson correlation would rank the same pair of users differently. Show the calculation.
18. User-based CF needs to compare the target user against every other user; item-based CF needs to compare the target item against every other item. In a system with 200 million users and 20,000 products (think Amazon), which comparison is cheaper to keep up to date, and why might that be the real reason item-based CF is more common in industry?

Dr. Ahmed El-Sayed

Eng. Youssef El-Ebiary Eng. Sohail Elganagy

Eng. Nour Mohamed Eng. Omar Abourida

Bonus (up to 10 extra points)

- Implement mean-centering explicitly for user-based CF (subtract each user's mean rating before predicting, add it back after) as a separate function, and compare its RMSE to plain Pearson-weighted prediction — they should be closely related; explain why.
- Generate a top-10 recommendation list (highest predicted ratings among items the user hasn't rated) for one user of your choice, and sanity-check it by hand: do the recommended titles' genres match that user's rating history?

7. Deliverables

- One Jupyter notebook (.ipynb) containing all code, outputs, plots, and your written answers as markdowns.
- Do not submit the dataset files — only your notebook. Assume the grader has ml-latest-small in a data/ folder next to it.

8. Allowed Tools and Academic Integrity

You may use pandas and numpy freely for data handling and basic math (means, dot products, norms, sqrt). You may not use scikit-learn's NearestNeighbors, pairwise_distances, or cosine_similarity, the Surprise library, LightFM, Implicit, or any other library that computes similarity or performs collaborative filtering for you. The point of this assignment is to build these mechanisms yourself; importing them defeats the purpose and will be treated as not completing the assignment, not as a shortcut.

- All use of Artificial Intelligence (AI) must be explicitly disclosed. You must clearly mark the exact beginning and end of any AI-generated sections using comments or citations. AI-generated content must remain completely separate and cannot be seamlessly interwoven with your original work. Any violation of this policy will result in the immediate rejection of your project.
- *Our Philosophy:* The goal of this policy is not to ban AI, but to maintain academic transparency and ensure you are achieving the required learning outcomes. AI is a valuable tool for accelerating your workflow once foundational skills are mastered, but it should not be used as a substitute for developing those skills in the first place.

Dr. Ahmed El-Sayed

Eng. Youssef El-Ebiary Eng. Sohail Elganagy

Eng. Nour Mohamed Eng. Omar Abourida

9. Grading Rubric

Section	Points	What earns full credit
A — Load & explore	5	Correct matrix construction, sensible sparsity/statistics reported
B — Similarity metrics	25	All three functions correct, verified against the worked example, co-rated handling correct
C — User-based CF	20	Correct neighbor selection and weighted prediction, sensible edge-case handling
D — Item-based CF	15	Correctly adapted from Part C, no unexplained code duplication
E — Evaluation	20	Correct train/test split per user, correct RMSE/MAE, full grid reported as a table
F — Analysis	15	Reasoning grounded in the student's own results and own worked example, not restated definitions
Bonus	up to 10	Either bonus task attempted with a clear, correct comparison

Final Notes

1. You should work in groups of exactly three, not finding a team is NOT an excuse.
2. You should deliver a python notebook attached with the comments in markdown cells, you should export it as pdf.
3. We will need both the pdf and the notebook in zipped file.
4. You should deliver with a naming scheme id _assigment.zip.
5. Delivery will be ignored if you didn't follow the naming scheme provided in 4, any one of the teamids can be used.
6. Any form of academic dishonesty, including but not limited to using AI tools blindly (such as ChatGPT or other code generation platforms), copying open-source code without proper attribution, or engaging in 'vibe coding' without genuine understanding, will be considered a serious violation and will be heavily penalized.

Dr. Ahmed El-Sayed

Eng. Youssef El-Ebiary Eng. Sohail Elganagy

Eng. Nour Mohamed Eng. Omar Abourida